

PASS PORTAL

Sistema de Gestión Migratoria

DOCUMENTACIÓN DEL PROYECTO INTEGRADOR

Desarrollador registrado	André Emanuel Yoj Gómez
Institución	Centro Educativo Técnico Kinal · Perito en Informática
Tecnologías	Node.js + TypeScript · Angular · MySQL
Estado	Aplicación funcional en desarrollo local
Versión / fecha	1.0.0 · 7 de octubre de 2026

Documento preparado a partir del README del proyecto y contrastado con la guía de especificaciones y rúbrica.

Los elementos marcados como PENDIENTES se conservan como pendientes porque no existe evidencia verificable en el material recibido.

Resumen ejecutivo de cumplimiento

Objetivo del documento. Presentar de forma profesional el análisis, diseño, implementación, seguridad, pruebas, manual y estado de entrega de Pass Portal, sin inventar evidencias que aún no existen.

Criterio de rúbrica	Valor	Estado documental actual
1. Problemática y alcance	10	Cubierto
2. Planificación y arquitectura	8	Parcial
3. Base de datos	12	Cubierto documentalmente
4. Backend Node.js + TypeScript	20	Mayormente cubierto
5. Frontend Angular	18	Mayormente cubierto
6. Integración y seguridad	10	Cubierto con pendientes
7. Diseño y UX	5	Parcial
8. Pruebas y calidad	7	Parcial
9. Despliegue	5	Pendiente
10. Manual y documentación	5	En construcción

Sistema de Gestión Migratoria

1. Introducción
2. Problemática mundial
3. Justificación
4. Objetivos
5. Población beneficiaria y usuarios
6. Alcance y limitaciones
7. Requisitos funcionales
8. Requisitos no funcionales
9. Historias de usuario y casos de uso
10. Arquitectura del sistema
11. Tecnologías utilizadas
12. Estructura del proyecto
13. Diseño de la base de datos
14. Modelo entidad-relación
15. Diccionario de datos
16. Backend
17. API REST y endpoints
18. Autenticación, autorización y seguridad
19. Frontend Angular
20. Módulos y funcionalidades
21. Identidad visual y paleta
22. Proceso principal
23. Dashboard e indicadores
24. Pruebas y control de calidad
25. Despliegue
26. Manual de usuario
27. Decisiones técnicas
28. Limitaciones conocidas
29. Conclusiones
30. Recomendaciones
31. Anexos y evidencias
32. Pendientes para completar la entrega

1. Introducción

Pass Portal es una aplicación web full stack para apoyar la gestión de información migratoria. La solución centraliza fichas ciudadanas, documentos, alertas de vencimiento, multas, arraigos, movimientos migratorios y solicitudes de movimiento.

El sistema fue construido con un backend Node.js y TypeScript, un frontend Angular y una base de datos relacional MySQL. La aplicación implementa autenticación mediante JWT, autorización por roles, formularios reactivos, validaciones en frontend y backend, consultas parametrizadas, manejo centralizado de errores y pruebas automatizadas.

El alcance actual está orientado a un entorno de demostración académica con datos ficticios. No pretende sustituir los sistemas oficiales de control migratorio ni conectarse con registros gubernamentales, bancos o servicios externos.

1.1 Estado actual verificable

- El frontend compila correctamente con Angular.
- El backend compila correctamente con TypeScript.
- La base de datos utiliza MySQL.
- El backend tiene pruebas automatizadas existentes.
- El frontend tiene pruebas automatizadas existentes.
- El sistema puede ejecutarse localmente en `http://localhost:4200` y `http://localhost:3000`.
- Las URLs públicas de producción todavía no están documentadas.

2. Problemática mundial

2.1 Problema identificado

La gestión migratoria requiere consultar información personal, documentos, vencimientos, restricciones, multas y movimientos de una misma persona. Cuando estos datos se encuentran dispersos o se revisan manualmente, aumenta el riesgo de:

- No detectar documentos próximos a vencer.
- Retrasar la atención de restricciones o arraigos.
- Dificultar la consulta del historial de una persona.
- Perder trazabilidad sobre multas y decisiones administrativas.
- Recibir solicitudes sin un proceso claro de revisión y aprobación.
- Consultar información sin diferenciar los permisos de cada tipo de usuario.

2.2 Problemática de alcance mundial

La movilidad humana y la migración internacional afectan a millones de personas y requieren procesos administrativos coordinados, trazables y respetuosos de la información personal. Pass Portal delimita esta problemática a la organización de información migratoria y al seguimiento de solicitudes, documentos y alertas en una plataforma web académica.

La aplicación no intenta resolver toda la migración mundial. Atiende un segmento concreto: la consulta y gestión organizada de información migratoria en una institución ficticia de control.

2.3 Relación con los Objetivos de Desarrollo Sostenible

Pendiente de validación académica: el equipo debe definir y justificar los ODS relacionados. Una relación posible, que debe ser aprobada por el docente, es:

- ODS 10: Reducción de las desigualdades, por facilitar procesos administrativos más claros para personas migrantes.
- ODS 16: Paz, justicia e instituciones sólidas, por promover trazabilidad, control de permisos y decisiones administrativas.

3. Justificación

Pass Portal se justifica porque concentra información que normalmente debe consultarse en diferentes procesos y ofrece una vista de prioridades basada en fechas de vencimiento. La solución permite que cada rol acceda solo a las funciones que necesita.

Para el ciudadano, el sistema proporciona un perfil, una ficha migratoria y un mecanismo formal para solicitar movimientos indicando origen, destino, fecha y motivo. Para el personal, permite revisar la solicitud, consultar la ficha relacionada y aprobar o rechazar el proceso. Para el administrador, incorpora la gestión de usuarios y roles.

La solución es viable para el alcance académico porque utiliza tecnologías requeridas por la guía, datos ficticios, un esquema reproducible y un flujo transaccional demostrable.

4. Objetivos

4.1 Objetivo general

Desarrollar una aplicación web full stack que organice información migratoria, genere alertas documentales y gestione solicitudes de movimiento mediante autenticación, autorización por roles y un proceso de revisión trazable.

4.2 Objetivos específicos

1. Implementar registro e inicio de sesión con contraseñas protegidas mediante bcrypt.
2. Proteger las rutas del frontend y los endpoints del backend según el rol autenticado.
3. Centralizar fichas ciudadanas, documentos, multas, restricciones y movimientos migratorios.
4. Clasificar documentos como vigentes, próximos a vencer o vencidos.
5. Permitir que los ciudadanos envíen solicitudes con país de origen, país de destino, fecha y motivo.
6. Permitir que personal y administradores revisen, aprueben o rechacen solicitudes.
7. Mostrar un dashboard con indicadores calculados desde la base de datos.
8. Incorporar validaciones, estados de carga, errores, éxito y listas vacías.
9. Proporcionar un script SQL único con esquema y datos de demostración.
10. Preparar una base técnica para futuras mejoras de despliegue y documentación.

5. Población beneficiaria y usuarios

5.1 Población beneficiaria

- Personas ciudadanas que necesitan consultar o completar su información migratoria.
- Personal encargado de revisar fichas, documentos, multas, restricciones y solicitudes.
- Administradores responsables de usuarios, roles y vinculación de fichas.

La población real, institución y cantidad esperada de usuarios no están especificadas en el repositorio.

Pendiente: completar con el alcance aprobado por el equipo y el docente.

5.2 Roles del sistema

Rol, Descripción, Permisos principales

USUARIO, Ciudadano vinculado a una ficha, Perfil, ficha propia, pago de demostración y solicitudes propias

PERSONAL, Personal operative, Dashboard, ciudadanos, fichas, multas, arraigos y solicitudes

ADMINISTRADOR, Administración del Sistema, Funciones del personal más gestión de usuarios y roles

6. Alcance y limitaciones

6.1 Incluido en el alcance

- Registro e inicio de sesión.
- Perfil editable.
- Gestión de roles.
- Fichas ciudadanas.
- Documentos y alertas de vencimiento.
- Multas y pago de demostración.
- Arraigos, bloqueos y restricciones.
- Historial de movimientos migratorios.
- Solicitudes de movimiento con aprobación o rechazo.
- Carga y visualización de fotografías.
- Dashboard operativo.
- Búsqueda y paginación de ciudadanos.
- Script SQL único de instalación.

6.2 Fuera del alcance

- Conexión con sistemas gubernamentales reales.
- Validación oficial de identidad o DPI.
- Integración con bancos o tarjetas.
- Pasarela de pagos real.
- Notificaciones por correo o SMS.
- Geolocalización o mapas en tiempo real.
- Aplicación móvil nativa.
- Firma digital.
- Despliegue productivo incluido en el repositorio.

7. Requisitos funcionales

- | Código | Requisito | Evidencia actual |
- | RF-01 | Registro e inicio de sesión | auth.router.ts, login y register Angular |
- | RF-02 | Hash seguro de contraseñas | bcryptjs en auth.service.ts |
- | RF-03 | Autenticación JWT | JWT firmado y middleware authenticate |
- | RF-04 | Tres roles con permisos diferentes | USUARIO, PERSONAL, ADMINISTRADOR |
- | RF-05 | Protección frontend y backend | guards Angular y requireRole |
- | RF-06 | Consulta y actualización de perfil | ruta /perfil y PUT /api/auth/me |
- | RF-07 | Entidades principales de negocio | ciudadanos, documentos, multas, arraigos, movimientos y solicitudes |
- | RF-08 | Operaciones de gestión | ciudadanos, multas, arraigos, usuarios y solicitudes |
- | RF-09 | Proceso transaccional | solicitud de movimiento y aprobación/rechazo |
- | RF-10 | Búsqueda, filtros y paginación | ciudadanos y alertas documentales |
- | RF-11 | Dashboard con datos reales | métricas, gráfico y alertas |
- | RF-12 | Validaciones en frontend/backend | Validators Angular y validadores de rutas |
- | RF-13 | Estados del proceso | pendiente, aprobada y rechazada |
- | RF-14 | Confirmación de acciones | pago, anulación y levantamiento de registros |
- | RF-15 | Estados visibles de interfaz | carga, error, éxito, vacío y deshabilitado |
- | RF-16 | Cierre de sesión | AuthService.logout() elimina sesión local |

Nota: el CRUD de algunas entidades es operativo y no todos los recursos exponen eliminación física. Las multas y restricciones utilizan cambios de estado, lo que evita eliminar historial administrativo.

8. Requisitos no funcionales

- | Categoría | Implementación o estado |
- | Seguridad | JWT, bcrypt, roles, CORS, Helmet, rate limit y consultas parametrizadas |
- | Usabilidad | Navegación por roles, mensajes, estados vacíos y acciones visibles |
- | Rendimiento | Consultas agrupadas y paginación frontend en listados |
- | Compatibilidad | Probado en entorno local con navegador integrado; compatibilidad adicional pendiente |
- | Mantenibilidad | Separación por routers, servicios, modelos, middleware y componentes |
- | Disponibilidad | Disponible localmente; despliegue público pendiente |
- | Accesibilidad | Labels, mat-error, aria-label, role="alert", foco visible y texto alternativo |
- | Privacidad | Datos demo ficticios; las fotografías requieren protección adicional en producción |

9. Historias de usuario y casos de uso

HU-01: Inicio de sesión

Como usuario registrado, quiero iniciar sesión con usuario y contraseña, para acceder a las funciones de mi rol.

Criterios:

- Las credenciales vacías muestran errores.
- Las credenciales incorrectas muestran un mensaje controlado.
- El backend entrega un JWT solo si la cuenta está activa.
- El usuario es redirigido según su rol.

HU-02: Completar ficha ciudadana

Como ciudadano, quiero completar mi DPI, nombres, apellidos, fecha de nacimiento, sexo y nacionalidad, para tener una ficha migratoria propia.

HU-03: Consultar alertas

Como personal, quiero ver documentos vencidos y próximos a vencer, para priorizar la gestión.

HU-04: Gestionar usuario

Como administrador, quiero crear, editar, activar o desactivar cuentas, para controlar el acceso al sistema.

HU-05: Solicitar movimiento

Como ciudadano, quiero indicar origen, destino, fecha y motivo, para solicitar un movimiento migratorio.

HU-06: Resolver solicitud

Como personal o administrador, quiero consultar la ficha del solicitante y aprobar o rechazar su solicitud, para controlar el proceso.

HU-07: Cargar fotografía

Como usuario autorizado, quiero cargar una fotografía JPG, PNG o WebP de máximo 5 MB, para identificar visualmente la ficha.

Caso de uso principal

1. El ciudadano inicia sesión.
2. Abre Mi información migratoria.
3. Completa o consulta su ficha.
4. Envía una solicitud con origen, destino, fecha y motivo.
5. El backend valida los datos y guarda el estado PENDIENTE.

6. El personal abre Solicitudes de movimiento.
7. Consulta los datos del ciudadano y abre su ficha.
8. Aprueba o rechaza la solicitud.
9. Si aprueba, se crea el movimiento en el historial.
10. El ciudadano consulta el resultado en su lista de solicitudes.

10. Arquitectura del sistema

Usuario del navegador --> Frontend Angular

Frontend Angular -> API Express Node.js TypeScript (HTTP JSON + JWT)

API Express Node.js TypeScript -> Middleware de autenticación y roles

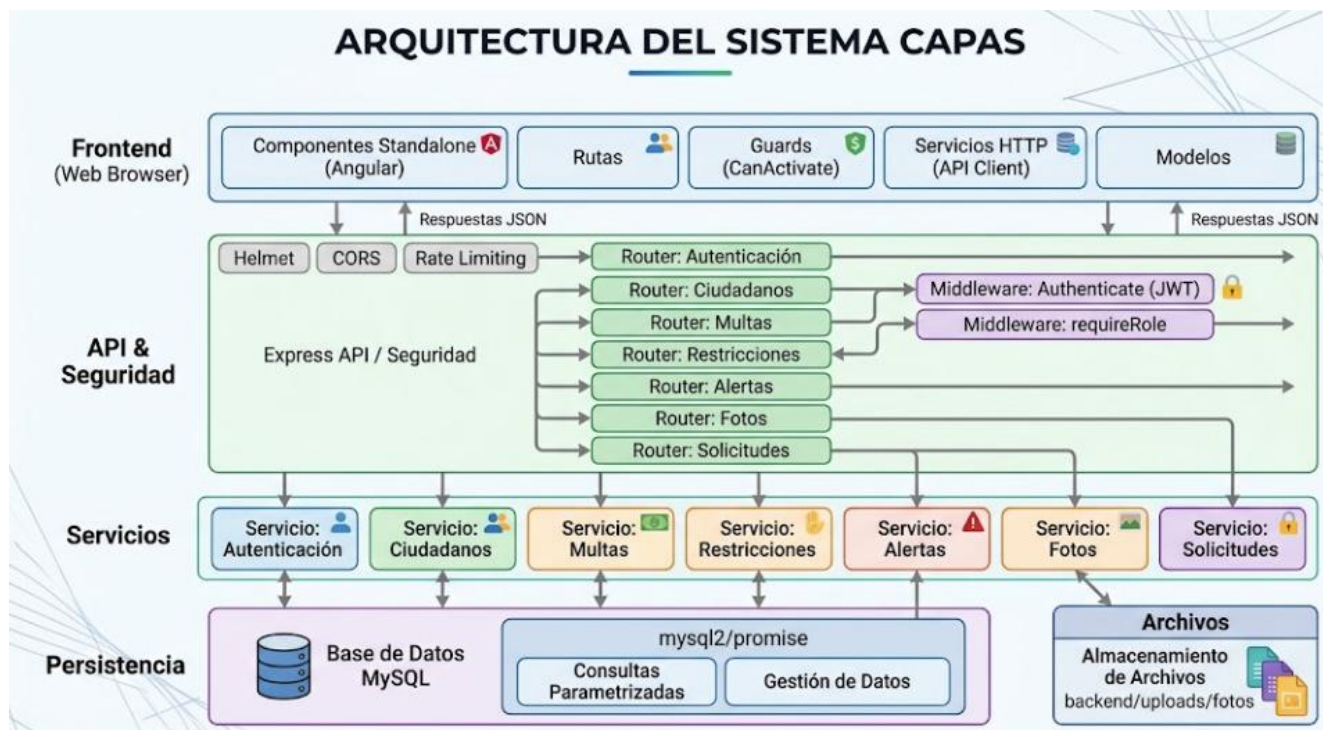
Middleware de autenticación y roles -> Servicios de dominio

Servicios de dominio -> MySQL

API Express Node.js TypeScript -> Archivos de fotografías

Capas

- Frontend: componentes standalone, rutas, guards, servicios HTTP y modelos.
- API: Express, routers por dominio y respuestas JSON.
- Seguridad: JWT, middleware authenticate, requireRole, Helmet, CORS y rate limit.
- Servicios: lógica de autenticación, ciudadanos, multas, restricciones, alertas, fotos y solicitudes.
- Persistencia: MySQL mediante mysql2/promise con consultas parametrizadas.
- Archivos: fotografías almacenadas en backend/uploads/fotos.



11. Tecnologías utilizadas

| Capa | Tecnología | Uso |

| Backend | Node.js 22+ | Runtime |

| Backend | TypeScript | Tipado y compilación |

| Backend | Express 5 | API REST |

| Backend | mysql2 | Acceso a MySQL |

| Backend | bcryptjs | Hash de contraseñas |

| Backend | jsonwebtoken | JWT |

| Backend | multer | Carga de fotografías |

| Backend | Helmet | Cabeceras de seguridad |

| Backend | cors | Política de origen cruzado |

| Backend | express-rate-limit | Límite de intentos de login |

| Frontend | Angular 22 | Aplicación web |

| Frontend | Angular Material/CDK | Componentes visuales |

| Frontend | RxJS | Flujos HTTP y reactividad |

| Base de datos | MySQL 8+ | Persistencia relacional |

| Paquetes | npm | Gestión del monorepositorio |

| Pruebas | Node test runner | Pruebas backend |

| Pruebas | Vitest mediante Angular | Pruebas frontend |

12. Estructura del proyecto

PassPortal/

```
├── backend/
│   ├── scripts/
│   ├── src/
│   │   ├── db/
│   │   │   └── setup.sql
│   │   ├── middleware/
│   │   ├── models/
│   │   ├── router/
│   │   ├── services/
│   │   └── utils/
│   └── test/
```

```
├── frontend/
│   ├── public/images/
│   └── src/app/
│       ├── components/
│       ├── models/
│       ├── services/
│       └── utils/
├── package.json
├── pnpm-workspace.yaml
├── pnpm-lock.yaml
├── README.md
└── Readme word.md
```

Los templates y estilos de los componentes Angular están definidos dentro de archivos .ts mediante templates inline.

13. Diseño de la base de datos

La base se llama pass_portal_IN5BM y utiliza InnoDB, claves primarias, claves foráneas, índices, restricciones CHECK, campos ENUM y marcas de tiempo.

Tablas principales

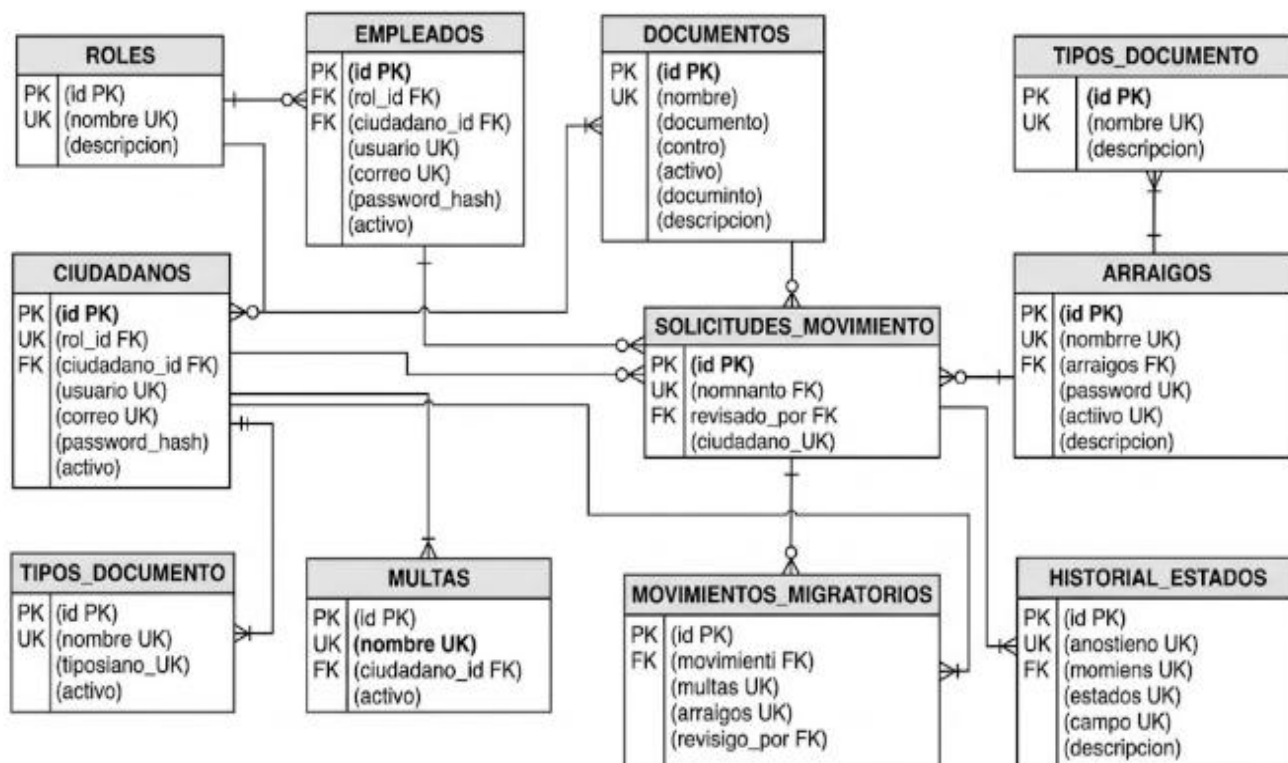
1. roles
2. empleados
3. ciudadanos
4. tipos_documento
5. documentos
6. multas
7. arraigos
8. movimientos_migratorios
9. solicitudes_movimiento
10. historial_estados

Relaciones principales

- Un rol puede pertenecer a muchos empleados.
- Una ficha ciudadana puede tener muchos documentos.
- Una ficha ciudadana puede tener muchas multas.
- Una ficha ciudadana puede tener muchas restricciones.

- Una ficha ciudadana puede tener muchos movimientos.
- Una ficha ciudadana puede tener muchas solicitudes de movimiento.
- Un empleado puede estar vinculado a una ficha ciudadana.
- Un empleado puede registrar o resolver operaciones.

14. Modelo entidad-relación



15. Diccionario de datos

15.1 roles

Campo	Tipo	Nulo	Restricción	Descripción
id	int	No	PK	Identificador único de la actividad
nombre	varchar(100)	No		Nombre de la actividad
descripcion	text	Si		Descripción detallada de la actividad
fecha_inicio	datetime	No		Fecha y hora de inicio de la actividad
fecha_fin	datetime	No		Fecha y hora de finalización de la actividad
estado	enum('Pendiente', 'En Progreso', 'Completada')	No		Estado actual de la actividad
usuario_id	int	No	FK a usuarios(id)	Identificador del usuario responsable de la actividad
proyecto_id	int	Si	FK a proyectos(id)	Identificador del proyecto al que pertenece la actividad

id	TINYINT UNSIGNED	No	PK, autoincremento	Identificador del rol
----	------------------	----	--------------------	-----------------------

nombre	VARCHAR(30)	No	Único	Nombre del rol
--------	-------------	----	-------	----------------

| descripcion | VARCHAR(150) | No | - | Descripción del permiso |

15.2 empleados

Campo	Tipo	Nulo	Restricción	Descripción
id	INT UNSIGNED	No	PK, autoincremento	Identificador del usuario
rol_id	TINYINT UNSIGNED	No	FK a roles	Rol asignado
nombre_completo	VARCHAR(120)	No	-	Nombre visible
usuario	VARCHAR(40)	No	Único	Usuario de acceso
correo	VARCHAR(120)	No	Único	Correo de acceso
password_hash	VARCHAR(255)	No	Hash bcrypt	Contraseña protegida
ciudadano_id	INT UNSIGNED	Sí	FK a ciudadanos	Ficha vinculada
activo	TINYINT(1)	No	Default 1	Estado de la cuenta
creado_en	TIMESTAMP	No	Default actual	Fecha de creación
actualizado_en	TIMESTAMP	No	Actualización automática	Última modificación

15.3 ciudadanos

Campo	Tipo	Nulo	Restricción	Descripción
id	INT UNSIGNED	No	PK, autoincremento	Identificador de ficha
dpi	VARCHAR(20)	No	Único	Identificación ciudadana
nombres	VARCHAR(80)	No	-	Nombres
apellidos	VARCHAR(80)	No	Índice por apellidos	Apellidos
fecha_nacimiento	DATE	No	No futura en API	Fecha de nacimiento
sexo	ENUM(M,F,X)	No	-	Sexo registrado
nacionalidad	VARCHAR(60)	No	-	Nacionalidad
telefono	VARCHAR(20)	Sí	-	Teléfono
correo	VARCHAR(120)	Sí	-	Correo de contacto
direccion	VARCHAR(255)	Sí	-	Dirección
foto_url	VARCHAR(255)	Sí	-	Ruta de fotografía
creado_en	TIMESTAMP	No	Default actual	Fecha de creación
actualizado_en	TIMESTAMP	No	Actualización automática	Última modificación

15.4 tipos_documento

Campo	Tipo	Nulo	Restricción	Descripción
id	TINYINT UNSIGNED	No	PK	Identificador
codigo	VARCHAR(30)	No	Único	Código del documento
nombre	VARCHAR(80)	No	-	Nombre visible

15.5 documentos

Campo	Tipo	Nulo	Restricción	Descripción
id	INT UNSIGNED	No	PK	Identificador
ciudadano_id	INT UNSIGNED	No	FK	Ficha propietaria
tipo_documento_id	TINYINT UNSIGNED	No	FK	Tipo de documento
numero	VARCHAR(40)	No	Único por tipo	Número documental
pais_emisor	VARCHAR(60)	No	-	País emisor
fecha_emision	DATE	No	-	Fecha de emisión
fecha_vencimiento	DATE	No	CHECK	Vencimiento
observaciones	VARCHAR(255)	Sí	-	Notas
creado_en	TIMESTAMP	No	Default actual	Creación
actualizado_en	TIMESTAMP	No	Automático	Modificación

15.6 multas

Campo	Tipo	Nulo	Restricción	Descripción
id	INT UNSIGNED	No	PK	Identificador
ciudadano_id	INT UNSIGNED	No	FK	Ciudadano relacionado
concepto	VARCHAR(200)	No	-	Motivo de la multa
monto	DECIMAL(10,2)	No	Mayor que cero	Monto
moneda	CHAR(3)	No	Default GTQ	Moneda
estado	ENUM	No	PENDIENTE/PAGADA/ANULADA	Estado
fecha_registro	DATE	No	-	Fecha de alta
fecha_pago	DATE	Sí	-	Fecha de pago
registrado_por	INT UNSIGNED	Sí	FK	Empleado que registra
creado_en	TIMESTAMP	No	Default actual	Creación
actualizado_en	TIMESTAMP	No	Automático	Modificación

15.7 arraigos

Campo	Tipo	Nulo	Restricción	Descripción
id	INT UNSIGNED	No	PK	Identificador
ciudadano_id	INT UNSIGNED	No	FK	Ciudadano relacionado
tipo	ENUM	No	Tres tipos de restricción	Tipo
motivo	VARCHAR(255)	No	-	Motivo
autoridad	VARCHAR(120)	No	-	Autoridad emisora
numero_expediente	VARCHAR(60)	Sí	-	Expediente

fecha_inicio	DATE	No	-	Inicio
fecha_fin	DATE	Sí	CHECK	Fin
activo	TINYINT(1)	No	Default 1	Estado
registrado_por	INT UNSIGNED	Sí	FK	Empleado
creado_en	TIMESTAMP	No	Default actual	Creación
actualizado_en	TIMESTAMP	No	Automático	Modificación

15.8 movimientos_migratorios

Campo	Tipo	Nulo	Restricción	Descripción
id	INT UNSIGNED	No	PK	Identificador
ciudadano_id	INT UNSIGNED	No	FK	Ciudadano
documento_id	INT UNSIGNED	Sí	FK	Documento asociado
tipo	ENUM(ENTRADA,SALIDA)	No	-	Tipo de movimiento
fecha_hora	DATETIME	No	-	Momento del movimiento
puesto_control	VARCHAR(100)	No	-	Puesto o fuente
pais_origen_destino	VARCHAR(60)	No	-	País relacionado
creado_en	TIMESTAMP	No	Default actual	Creación

15.9 solicitudes_movimiento

Campo	Tipo	Nulo	Restricción	Descripción
id	INT UNSIGNED	No	PK	Identificador de solicitud
ciudadano_id	INT UNSIGNED	No	FK	Solicitante
pais_origen	VARCHAR(60)	No	Diferente de destino en API y DB	Origen
pais_destino	VARCHAR(60)	No	Diferente de origen en API y DB	Destino
fecha_solicitada	DATE	No	Formato AAAA-MM-DD	Fecha prevista
motivo	VARCHAR(255)	No	-	Motivo
estado	ENUM	No	PENDIENTE/APROBADA/RECHAZADA	Estado
comentario_resolucion	VARCHAR(255)	Sí	-	Comentario del revisor
revisado_por	INT UNSIGNED	Sí	FK	Empleado revisor
revisado_en	DATETIME	Sí	-	Fecha de resolución
creado_en	TIMESTAMP	No	Default actual	Creación
actualizado_en	TIMESTAMP	No	Automático	Modificación

15.10 historial_estados

Campo	Tipo	Nulo	Restricción	Descripción
id	BIGINT UNSIGNED	No	PK	Identificador de auditoría
ciudadano_id	INT UNSIGNED	No	FK	Ciudadano
entidad	ENUM	No		Ciudadano/documento/multa/arraigo Entidad afectada
entidad_id	INT UNSIGNED	Sí -		ID de entidad
estado_anterior	VARCHAR(30)	Sí -		Estado previo
estado_nuevo	VARCHAR(30)	No -		Nuevo estado
detalle	VARCHAR(255)	Sí -		Descripción
empleado_id	INT UNSIGNED	Sí	FK	Responsable
creado_en	TIMESTAMP	No	Default actual	Fecha

16. Backend

16.1 Organización

El backend se divide en:

- router: rutas HTTP por dominio.
- services: reglas de negocio y consultas.
- models: tipos TypeScript.
- middleware: autenticación y autorización.
- utils: errores, respuestas y rutas.
- db: conexión MySQL y script SQL.
- test: pruebas automatizadas.

16.2 Manejo de respuestas

Las respuestas exitosas utilizan una estructura ok/data. Los errores utilizan códigos HTTP y mensajes controlados mediante `HttpError` y `errorHandler`. No se envían trazas internas al cliente.

16.3 Proceso transaccional

La aprobación de una solicitud se ejecuta dentro de una transacción MySQL:

1. Se bloquea la solicitud con `FOR UPDATE`.
2. Se verifica que siga en estado `PENDIENTE`.
3. Se actualiza a `APROBADA` o `RECHAZADA`.
4. Si se aprueba, se inserta el movimiento histórico.

5. Se confirma la transacción.
6. Una solicitud ya resuelta no puede resolverse nuevamente.

17. API REST y endpoints

Todas las rutas salvo health y login requieren Authorization: Bearer \<token>.

Autenticación

Método	Ruta	Roles	Descripción
POST	/api/auth/login	Público	Inicia sesión y devuelve JWT
POST	/api/auth/register	Público	Registra usuario normal
GET	/api/auth/me	Todos autenticados	Consulta sesión
GET	/api/auth/me/ficha	Usuario autenticado	Consulta ficha propia
PUT	/api/auth/me/ficha	USUARIO	Crea o actualiza ficha propia
PUT	/api/auth/me	Todos autenticados	Actualiza perfil

Ciudadanos y ficha

Método	Ruta	Roles	Descripción
GET	/api/ciudadanos?q=	PERSONAL, ADMINISTRADOR	Busca ciudadanos
GET	/api/ciudadanos/:id	PERSONAL, ADMINISTRADOR	Consulta ficha completa
PUT	/api/ciudadanos/:id	PERSONAL, ADMINISTRADOR	Actualiza contacto
POST	/api/ciudadanos/:id/foto	Todos según permiso	Sube fotografía

Alertas y dashboard

Método	Ruta	Roles	Descripción
GET	/api/alertas	PERSONAL, ADMINISTRADOR	Lista alertas
GET	/api/alertas/resumen	PERSONAL, ADMINISTRADOR	Métricas del dashboard

Multas y restricciones

Método	Ruta	Roles	Descripción
POST	/api/ciudadanos/:id/multas	PERSONAL, ADMINISTRADOR	Crea multa
GET	/api/ciudadanos/:id/multas	Autenticado	Lista multas
PATCH	/api/multas/:id/pagar	Usuario relacionado, personal o admin	Paga demo
PATCH	/api/multas/:id/anular	PERSONAL, ADMINISTRADOR	Anula multa
POST	/api/ciudadanos/:id/arraigos	PERSONAL, ADMINISTRADOR	Crea restricción
GET	/api/ciudadanos/:id/arraigos	Autenticado	Lista restricciones

| PATCH | /api/arraigos/:id/levantar | PERSONAL, ADMINISTRADOR | Levanta restricción |

Solicitudes de movimiento

| Método | Ruta | Roles | Descripción |

| POST | /api/solicitudes-movimiento | USUARIO | Crea solicitud propia |

| GET | /api/solicitudes-movimiento/mias | USUARIO | Lista solicitudes propias |

| GET | /api/solicitudes-movimiento?estado= | PERSONAL, ADMINISTRADOR | Lista solicitudes operativas |

| PATCH | /api/solicitudes-movimiento/:id/resolver | PERSONAL, ADMINISTRADOR | Aprueba o rechaza |

Códigos usados

- 200: consulta o actualización exitosa.
- 201: creación exitosa.
- 400: entrada inválida.
- 401: falta autenticación o token inválido.
- 403: rol sin permiso.
- 404: recurso inexistente.
- 409: conflicto, por ejemplo solicitud ya resuelta.

18. Autenticación, autorización y seguridad

Autenticación

- El usuario inicia sesión con usuario y contraseña.
- bcryptjs genera y compara hashes con factor 12.
- JWT contiene usuario, rol y vínculo ciudadano.
- El frontend conserva el token en el servicio de autenticación.
- Cerrar sesión elimina la información local de autenticación.

Autorización

- Angular utiliza authGuard, roleGuard y adminGuard.
- Express valida el JWT con authenticate.
- requireRole limita cada endpoint en el servidor.
- El servidor no confía únicamente en los guards del frontend.

Medidas implementadas

- Consultas parametrizadas con mysql2.
- CORS configurado.
- Helmet habilitado.
- Rate limit para login.
- Límite de JSON de 1 MB.
- Límite de fotografía de 5 MB.

- Validación de tipos de imagen.
- Variables sensibles en .env.
- Mensajes de error controlados.

Riesgos pendientes

- Las fotos se publican actualmente como archivos estáticos.
- La URL de foto incluye el DPI en el nombre del archivo.
- Debe existir un .env.example sin secretos para la entrega.
- Debe verificarse que ningún .env real se publique en Git.

19. Frontend Angular

El frontend utiliza componentes standalone, rutas lazy-loaded, formularios reactivos y servicios HTTP.

Rutas principales

Ruta	Acceso
/login	Público
/register	Público
/perfil	Todos autenticados
/mi-informacion	USUARIO
/mi-informacion/ficha	USUARIO
/dashboard	PERSONAL, ADMINISTRADOR
/ciudadanos	PERSONAL, ADMINISTRADOR
/ciudadanos/:id	PERSONAL, ADMINISTRADOR
/solicitudes	PERSONAL, ADMINISTRADOR
/admin/usuarios	ADMINISTRADOR

Estados de interfaz

- Cargando: barras y botones deshabilitados.
- Error: mensajes visibles y opción de reintentar.
- Éxito: actualización de tarjetas, listas o snackbar.
- Vacío: mensajes para listas sin registros.
- Deshabilitado: acciones bloqueadas durante operaciones.
- Validación: mensajes mat-error específicos por campo.

20. Módulos y funcionalidades

Login y registro

Permiten autenticación, registro público de cuentas normales, validación de campos y mensajes de error.

Perfil

Permite editar nombre, correo y contraseña opcional.

Mi información migratoria

Permite completar la ficha propia y enviar solicitudes de movimiento. Muestra el estado de cada solicitud.

Ciudadanos

Permite buscar por nombre, apellido o DPI y navegar a la ficha completa.

Ficha migratoria

Muestra fotografía, documentos, vencimientos, multas, restricciones y movimientos.

Usuarios y roles

El administrador puede crear, editar, activar, desactivar y vincular cuentas con fichas por DPI.

Dashboard

Muestra vencidos, próximos vencimientos, vigentes, multas pendientes, arraigos activos, distribución documental y alertas filtrables.

Solicitudes de movimiento

El personal y administrador consultan solicitudes, ven origen/destino, consultan ficha y aprueban o rechazan.

21. Identidad visual y paleta

La interfaz utiliza una identidad sobria y operativa relacionada con gestión institucional.

| Elemento | Valor | Uso |

| Fondo principal | #F3EFEA | Fondo general |

| Superficie | #FAF8F5 | Tarjetas y formularios |

| Primario vino | #72393F | Acciones, navegación y jerarquía |

| Primario hover | #5A2C31 | Estados hover |

| Texto principal | #332E2B | Títulos y datos |

| Texto secundario | #7A6F68 | Descripciones |

| Texto tenue | #A0938A | Ayudas y metadatos |

| Error | #B71C1C | Vencidos y errores |

| Éxito | #2E7D32 | Vigentes y aprobaciones |

| Advertencia | #D69E2E | Próximos vencimientos |

22. Proceso principal del sistema

El proceso principal es la solicitud y resolución de un movimiento migratorio.

mermaid

[*] --> PENDIENTE: Ciudadano envía solicitud

PENDIENTE --> APROBADA: Personal o admin aprueba

PENDIENTE --> RECHAZADA: Personal o admin rechaza

APROBADA --> MovimientoRegistrado: Se crea historial migratorio

RECHAZADA --> [*]

MovimientoRegistrado --> [*]

La aprobación y el registro del movimiento se realizan en la misma transacción para evitar que una solicitud quede aprobada sin movimiento histórico.

23. Dashboard e indicadores

El dashboard consulta información real de MySQL mediante `/api/alertas/resumen`.

Indicadores

- Documentos vencidos.
- Documentos por vencer en 30 días.
- Documentos por vencer en 60 días.
- Documentos por vencer en 90 días.
- Documentos vigentes.
- Multas pendientes.
- Arraigos activos.
- Total de documentos.

Visualizaciones

- Tarjetas métricas con estados rojo, ámbar y verde.
- Gráfico circular de distribución documental.
- Lista de atención prioritaria.
- Tabla de alertas documentales.
- Filtros por estado, tipo, rango y texto.
- Paginación y ordenamiento de la tabla.

24. Pruebas y control de calidad

24.1 Pruebas automatizadas backend

Archivos existentes:

- backend/test/alert-classifier.test.ts
- backend/test/auth.middleware.test.ts
- backend/test/auth.validation.test.ts

Resultado verificado: 3 pruebas aprobadas.

24.2 Pruebas automatizadas frontend

Archivo existente:

- frontend/src/app/app.spec.ts

Resultado verificado: 2 pruebas aprobadas.

24.3 Validaciones ejecutadas

- pnpm --filter backend typecheck
- pnpm --filter backend build
- pnpm --filter backend test
- pnpm --filter frontend build
- pnpm --filter frontend test -- --watch=false
- Diagnóstico global del editor sin errores.

24.4 Casos funcionales recomendados para evidencia

| Caso | Resultado esperado |

| Login correcto | Acceso según rol |

| Login incorrecto | Mensaje de credenciales inválidas |

| Registro con correo inválido | Validación visible |

| Contraseña corta | Rechazo frontend/backend |

| Ruta sin token | HTTP 401 |

| Ruta con rol incorrecto | HTTP 403 |

| DPI inválido | HTTP 400 y mensaje comprensible |

| Solicitud sin origen | Formulario inválido |

| Solicitud ya resuelta | HTTP 409 |

| Aprobación correcta | Solicitud aprobada y movimiento creado |

| Foto mayor de 5 MB | Error de carga |

| Foto con formato inválido | Error de formato |

| Búsqueda de ciudadanos | Resultados filtrados |

| Lista vacía | Estado vacío visible |

| Logout | Token eliminado y retorno a login |

Pendiente: capturar resultado real de cada caso en un plan de pruebas entregable.

25. Despliegue

Desarrollo local

Requisitos:

- Node.js 22 o posterior.
- pnpm 10 o posterior.
- MySQL 8 o posterior.

Instalación:

powershell

pnpm install

Base de datos:

1. Configurar backend/.env.
2. Ejecutar backend/src/db/setup.sql en MySQL Workbench.
3. Usar pnpm --filter backend migrate\:\movement-requests solo si la base ya existía antes de la tabla de solicitudes.

Ejecución:

powershell

pnpm dev\:\backend

pnpm dev\:\frontend

URLs locales:

- Frontend: http\://localhost:4200
- Backend: http\://localhost:3000
- Health: http\://localhost:3000/api/health

, URLs HTTPS, variables configuradas, base remota y evidencia de verificación.

26. Manual de usuario

26.1 Registro

1. Abrir /register.
2. Escribir nombre completo.
3. Escribir usuario.
4. Escribir correo válido.
5. Crear contraseña de mínimo 10 caracteres.
6. Presionar Crear cuenta.

26.2 Inicio de sesión

1. Abrir /login.
2. Escribir usuario y contraseña.
3. Presionar Ingresar.
4. El sistema muestra las opciones disponibles para el rol.

26.3 Ciudadano: completar ficha

1. Entrar a Mi información migratoria.
2. Completar DPI, nombres, apellidos, fecha, sexo y nacionalidad.
3. Agregar teléfono y dirección si corresponde.
4. Presionar Guardar ficha.
5. Abrir Ver mi ficha completa.

26.4 Ciudadano: solicitar movimiento

1. Entrar a Mi información migratoria.
2. Escribir país de origen.
3. Escribir país de destino.
4. Seleccionar fecha prevista.
5. Escribir el motivo.
6. Presionar Enviar solicitud.
7. Consultar el estado en Mis solicitudes.

26.5 Personal: revisar solicitud

1. Iniciar sesión con una cuenta PERSONAL.
2. Abrir Solicitudes de movimiento.
3. Filtrar por estado.
4. Revisar ciudadano, DPI, nacionalidad, origen, destino, fecha y motivo.
5. Presionar Ver ficha para revisar documentos y restricciones.
6. Presionar Aprobar o Rechazar.
7. Confirmar la acción.

26.6 Personal: revisar dashboard

1. Abrir Panel de alertas.
2. Revisar las tarjetas de métricas.
3. Consultar la distribución documental.
4. Filtrar alertas por estado, documento, rango o texto.
5. Abrir una ficha desde la tabla.

26.7 Administrador: gestionar usuarios

1. Iniciar sesión con rol ADMINISTRADOR.
2. Abrir Usuarios.
3. Crear usuario con nombre, usuario, correo, contraseña y rol.
4. Opcionalmente vincular DPI de una ficha.
5. Editar, activar o desactivar cuentas.

26.8 Cargar fotografía

1. Abrir una ficha autorizada.
2. Presionar Cambiar foto.
3. Seleccionar JPG, PNG o WebP.
4. Verificar que no supere 5 MB.
5. Esperar el mensaje de carga.
6. Confirmar que la nueva imagen aparece en la ficha.

26.9 Cerrar sesión

Presionar Cerrar sesión en la barra superior. El sistema elimina la sesión local y vuelve al login.

27. Decisiones técnicas

MySQL

Se eligió MySQL porque es un motor relacional adecuado para relaciones entre ciudadanos, documentos, sanciones, solicitudes y auditoría. La aplicación usa claves foráneas, índices y restricciones.

SQL directo parametrizado

Se utiliza mysql2 en lugar de un ORM. Esta decisión reduce dependencias y mantiene consultas explícitas. Los valores se envían mediante parámetros para evitar concatenación insegura.

JWT

JWT permite validar la sesión en cada petición sin almacenar sesiones de servidor. El servidor continúa validando el rol, por lo que el frontend no es la única barrera.

Componentes standalone

Angular utiliza componentes standalone y carga lazy de pantallas para mantener los módulos separados y reducir el acoplamiento.

Estados en lugar de eliminación física

Multas, restricciones y solicitudes conservan su registro y cambian de estado. Esto facilita la trazabilidad de decisiones.

Script SQL único

backend/src/db/setup.sql contiene esquema, datos base, datos demo y cuentas de prueba en un solo archivo copiable.

28. Limitaciones conocidas

1. No existe despliegue público documentado.
2. No existe URL HTTPS de producción.
3. No existe colección Postman/Insomnia ni OpenAPI en el repositorio.
4. No existe un diccionario de datos separado; este documento lo incorpora.
5. No existe un diagrama ER como imagen; se proporciona Mermaid.
6. El plan de pruebas con evidencias todavía debe elaborarse.
7. La suite frontend es mínima y cubre principalmente el componente raíz.

8. Las fotografías se sirven como recursos estáticos públicos.
9. Los nombres de fotografías incluyen DPI y deberían usar identificadores opacos en producción.
10. El pago de multas es solamente una demostración.
11. No hay integración con correo, SMS, mapas o servicios gubernamentales.
12. La compatibilidad formal en dos navegadores debe documentarse con capturas.
13. Los integrantes y responsabilidades del equipo no están registrados.
14. El alcance de beneficiarios reales y datos institucionales debe ser validado.

29. Conclusiones

Pass Portal cumple la base técnica de una aplicación web full stack funcional: integra Angular, Node.js con TypeScript y MySQL; implementa autenticación, roles, validaciones, estados de proceso, dashboard y un flujo transaccional de solicitudes.

El proyecto delimita una necesidad real y la convierte en procesos demostrables. Su principal flujo de negocio permite que un ciudadano solicite un movimiento y que personal autorizado consulte la ficha, revise la información y apruebe o rechace la solicitud.

30. Recomendaciones

1. Crear .env.example sin credenciales.
2. Publicar frontend, backend y MySQL en un proveedor compatible.
- 3\.. Agregar documentación OpenAPI o colección Postman.
4. Proteger fotos con autorización y nombres opacos.
5. Agregar pruebas de componentes y flujos principales del frontend.
6. Crear pruebas de integración para solicitud, aprobación y rechazo.
7. Incorporar auditoría específica para solicitudes de movimiento.
8. Documentar el contraste de colores con evidencia WCAG.
9. Preparar capturas de cada rol y cada flujo.
10. Definir integrantes, responsabilidades y commits.
11. Añadir un plan de pruebas con resultados esperados y obtenidos.
12. Validar el script SQL desde una base limpia antes de la entrega.
13. Generar una imagen del modelo entidad-relación.
14. Revisar que las credenciales demo no se usen en producción.

31. Anexos y evidencias

Anexo A: credenciales demo locales

| Usuario | Rol | Contraseña |

| usuario1 | USUARIO | Usuario2026@ |

| personal1 | PERSONAL | Personal2026@ |

| administrador1 | ADMINISTRADOR | Admin2026@ |

Estas credenciales son únicamente para desarrollo local y no deben publicarse como credenciales de producción.

Anexo B: comandos de verificación

powershell

pnpm install

pnpm --filter backend typecheck

pnpm --filter backend build

pnpm --filter backend test

pnpm --filter frontend build

pnpm --filter frontend test -- --watch=false

Anexo C: archivos de evidencia disponibles

- *README.md: instalación, ejecución, roles, endpoints y recorrido.*
- *backend/src/db/setup.sql: instalación completa de base de datos.*
- *backend/test/: pruebas backend.*
- *frontend/src/app/app.spec.ts: pruebas frontend.*
- *frontend/src/app/components/: pantallas Angular.*
- *backend/src/router/: endpoints REST.*
- *backend/src/services/: lógica de negocio.*

FIN DEL DOCUMENTO